TRUEVOICE / FORENSIC AUDIO BUREAU

Project & *presentation guide.*

A complete handover for running, demoing, and defending the audio deepfake detection system — written for a Windows machine and a viva-style presentation.

Vol.01 / Issue №04 Build 0.4.0 For research & educational use

TrueVoice — Presentation & Project Guide

This document is written for the project owner (running on Windows) and any collaborators. It explains everything that was changed, why, how the system works end-to-end, and how to walk an examiner through it.

---

1. Windows setup (one-time)

1.1 Install prerequisites

1. **Python 3.10 – 3.12** from python.org (tick "Add to PATH" during install).
2. **ffmpeg** — needed for the URL/YouTube tab. - Easiest: `winget install Gyan.FFmpeg` (Windows 10/11) - Or: download from https://www.gyan.dev/ffmpeg/builds/, extract, add `bin\` to PATH. - Verify in PowerShell: `ffmpeg -version`
3. **yt-dlp** — installed automatically with `pip install -r requirements.txt`. Verify: `yt-dlp --version`.
4. **Git** (only if you'll clone or share via Git).

1.2 Create virtual env and install

Open **PowerShell** in the project folder:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1            # macOS/Linux: source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

If PowerShell blocks the activation script: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.

1.3 First run

```
python app.py
```

- First request downloads the HuggingFace model (~370 MB, cached at `%USERPROFILE%\.cache\huggingface`).
- Default URL is **http://127.0.0.1:5001** (port 5000 collides with macOS AirPlay; 5001 works on every platform).

- Override port with `set PORT=8000 && python app.py`.

1.4 Notes for Windows users specifically

- **GPU**: TensorFlow GPU on Windows requires CUDA (NVIDIA only). CPU works fine for inference and small fine-tunes.
- **PyTorch device**: auto-selects `cuda` (NVIDIA), else CPU. (MPS is Mac-only.)
- **Path separators**: code uses `pathlib`, so it's portable.
- **Long path names**: enable in Windows if you hit `MAX_PATH` errors when extracting the dataset.
- **Audio mic recording**: works in Chrome / Edge / Firefox. Make sure microphone permissions are granted to localhost.

---

2. What changed (before → after)

2.1 Original state

- `app.py` — Flask server with hardcoded Windows path to a Keras model and broken preprocessing alignment.
- `preprocess_audio.py` — offline batch preprocessor (5 s, trim silence, normalise).
- One trained Keras "fusion" CNN that took two inputs (mel-spectrogram + raw waveform) of a 3-second clip.
- A static Bootstrap "Strategy" template stitched in for the UI.
- `dataset/real` (2,525 wavs) + `dataset/fake` (10,660 wavs) — class-imbalanced training data.
- No tests, no logging, no config, no model evaluation.
- Symptom: detector worked on training data, failed on YouTube and external clips.

2.2 New state

File	Role
`app.py`	Flask app · 8 JSON endpoints · feedback loop · spectrogram cache
`detector.py`	Inference engine: HF model + Keras model ensemble · chunked predict · saliency · spectrogram render
`ingestion.py`	yt-dlp wrapper: YouTube/URL → wav
`augment.py`	Audio perturbations (noise/telephone/pitch/reverb/speed) for robustness panel
`history.py`	SQLite store of last 50 analyses
`evaluate.py`	Offline eval on `dataset/` with ablation across HF / Keras / ensemble
`evaluate_urls.py`	Out-of-distribution eval on a CSV of URLs
`train.py`	Fine-tune the Keras model on `dataset/` + `feedback/` with class weighting
`templates/index.html`	Single-page UI: Upload · Record · URL · Compare tabs · result dossier
`static/css/app.css`	Editorial-forensic theme (Instrument Serif + JetBrains Mono)
`static/js/app.js`	Drag-drop · WaveSurfer · timeline canvas · feedback buttons
`requirements.txt`	Pinned deps: Flask · transformers · torch · tensorflow · librosa · yt-dlp · scikit-learn

2.3 The feature list

1. **Pretrained transformer detector** — Wav2Vec2 model from HuggingFace, generalises to external clips.
2. **Ensemble with the locally-trained Keras model** — combine the user's CNN with the pretrained model.
3. **Live mic recording** — `MediaRecorder` in-browser; analyse a 5-30 s spoken sample.
4. **YouTube / URL ingestion** — paste a link, server-side `yt-dlp` extracts audio.
5. **Side-by-side comparison** — analyse two clips together (real-vs-cloned demos).
6. **Per-chunk timeline chart** — fake probability across overlapping 5 s windows + saliency strip.
7. **Mel-spectrogram visualisation** — with translucent per-chunk verdict overlay.
8. **Robustness panel** — apply noise / telephone-band / pitch / reverb / speed perturbation, see Δpp.
9. **History drawer** — last 20 analyses persisted to SQLite.
10. **JSON / CSV export** — full analysis report download.
11. **Feedback loop** — "Flag as REAL" / "Flag as FAKE" buttons append to `feedback/{real,fake}/`.
12. **Retraining script** — `python train.py` fine-tunes the Keras model on `dataset/ + feedback/`.
13. **Evaluation harness** — `evaluate.py` and `evaluate_urls.py` produce metrics tables for the report.

---

3. How the system works (talking points)

3.1 The detection pipeline

```
upload → librosa loads audio @ 16 kHz mono
       → split into overlapping 5 s windows (50 % overlap)
       → for each window:
            – Wav2Vec2 (HuggingFace) → softmax → P(fake)
            – [optional] Keras fusion CNN → sigmoid → P(fake)
       → blend per-chunk: ens = (1 – W)·HF + W·Keras
       → aggregate across chunks: top-K mean (default K=3)
       → if ens ≥ threshold → FAKE; else REAL
       → render mel-spectrogram → PNG
       → compute gradient saliency (dHF_logit/dx) per chunk
       → return JSON to UI
```

3.2 The two models

	Wav2Vec2 (HF)	Fusion CNN (your trained)
Architecture	Self-supervised transformer, raw waveform input	Two-branch CNN: mel-spec + raw waveform
Training data	ASVspoof-class data (millions of TTS/clean speech samples)	Your 13K-sample dataset
Strength	Generalises to unseen TTS, codecs, mics	Excellent on training distribution
Weakness	Slightly weaker on in-distribution data	Overfits to training distribution
In your eval	91.7 % accuracy / 6.7 % EER	96.7 % accuracy / 3.3 % EER

The point of the ensemble: **trade a tiny in-distribution loss for huge out-of-distribution gain**. On YouTube clips your CNN alone would tank because YouTube has music and codec compression that your dataset doesn't contain. The HF model fills that gap.

3.3 The feedback / retraining loop

```
user uploads clip → detector predicts
user disagrees → clicks "Flag as REAL" or "Flag as FAKE"
clip saved to feedback/real/<uuid>.wav  or  feedback/fake/<uuid>.wav
…repeat for many clips…
operator runs:  python train.py
                → loads existing model/fusion_model_final.keras
                → reads dataset/ + feedback/ → 3 s preprocessing
                → fine-tunes with class-weighted BCE + early stopping
                → saves model/fusion_model_final_v{N}.keras
restart server → ensemble now uses the updated model
```

Why this matters for the presentation: active learning / human-in-the-loop ML is a hot research area. You can show the live workflow.

3.4 The chunked-aggregation trick

Audio detectors that work on fixed windows are easy to fool with a long clip that has just a few synthetic moments. The fix:

- Split into overlapping windows so events near boundaries aren't missed.
- Compute fake-prob per window.
- Aggregate with **top-K mean** instead of plain mean: average the K most-suspicious windows. Catches local fakery a plain mean would dilute.
- A line chart in the UI shows fake-prob over time, so a viewer can literally see *where* the suspicion lives.

---

4. Concepts you should be ready to explain in the viva

4.1 Audio fundamentals

- **Sample rate (16 kHz)** — speech bandwidth is roughly 300 Hz – 3.4 kHz; 16 kHz captures it with margin. Standard in speech ML.
- **Mel-spectrogram** — a frequency-vs-time image where frequency is mapped to the mel scale (perceptually linear for humans). 128 mels = 128 frequency bins.
- **n_fft / hop_length** — STFT window size and stride; here 2048 / 512.
- **Mono** — single-channel audio; we discard stereo to halve compute and avoid channel mismatch.

4.2 Models

- **Wav2Vec2** — Facebook AI, 2020. Learns speech representations by masking parts of raw audio and reconstructing them (self-supervised). Pre-trained on huge unlabelled corpora; fine-tuned for downstream tasks like ASR or — here — deepfake detection.
- **Fusion CNN** — your model: a 2D Conv branch over the spectrogram + a 1D Conv branch over the raw waveform → concat → dense → sigmoid. The raw branch catches phase/timing artefacts; the spec branch catches frequency artefacts.
- **Sigmoid vs softmax** — sigmoid for binary, softmax for multi-class. Your CNN uses sigmoid → one scalar output. The HF model uses softmax → 2-d probability vector; we read the "fake" index.

4.3 Metrics (be ready for these — they go in the report)

- **Accuracy** — (TP + TN) / total. Easy but misleading if classes imbalanced.
- **Precision** = TP / (TP + FP). Of clips flagged FAKE, how many were truly fake?
- **Recall** = TP / (TP + FN). Of all true fakes, how many did we catch?
- **F1** — harmonic mean of precision and recall.

- **EER (Equal Error Rate)** — the threshold-free metric for spoof detection. The threshold where FPR = FNR. Lower is better. ASVspoof papers all report this.
- **Confusion matrix** — TP / FP / TN / FN. Good visual for the slides.

4.4 Generalisation / distribution shift

The single most important concept in your project. The models are trained on a particular sound (clean studio speech for "real", a particular TTS for "fake"). When you run the detector on YouTube — different mics, codec compression, music, multiple speakers — the input distribution **differs from training**. Even a 99 % accurate model can fail.

Mitigations applied: - Pretrained Wav2Vec2 model (broader training distribution → better generalisation). - Ensemble (cancels per-model weaknesses). - Top-K aggregation (lets a small suspicious segment trigger the alarm). - Robustness panel (lets you measure how the verdict shifts under perturbation — exactly the kind of analysis you'll be asked about).

4.5 Active learning / human-in-the-loop

The flag → retrain loop is **active learning**. You're using the model itself to surface confusing samples (the ones the user disagrees with), and feeding those back into training. Two reasons this is well-known to work:

1. The most informative training samples are the ones near the decision boundary — exactly the ones the user is likely to flag.
2. Models drift over time as new TTS systems appear; periodic feedback-driven fine-tunes keep them current.

---

5. Demo flow (10-minute presentation)

A suggested walkthrough that covers the most impressive features in a stable order.

Slide 0 — Setup before the demo

1. Activate the venv. Start the server: `python app.py`.
2. Wait for `ready — http://127.0.0.1:5001`.
3. Have these tabs open in the browser: - The live UI at http://127.0.0.1:5001 - A terminal showing `app.py` logs (so you can show per-chunk numbers live) - The folder explorer with `feedback/` visible
4. Have **2–3 audio clips ready on the desktop**: one obvious deepfake, one genuine speech, one ambiguous clip (a podcast with intro music).

Step 1 — The problem (1 min)

> *"Audio deepfakes are weaponised. We built a forensic listening console that tells you whether a clip is real or AI-generated, and shows why."*

Open the UI; gesture at the editorial layout. Point out the four input modes.

Step 2 — Upload a known deepfake (2 min)

Drag-drop a deepfake into the upload zone. Click **Analyse**.

While inference runs, point at the **server log** in the terminal:

```
labels: {0: 'BONAFIDE', 1: 'SPOOF'}
fake_idx resolved to: 1 (SPOOF)
hf_avg=['0.821', ...]  keras=['0.910', ...]  blended=['0.866', ...]
Predict: chunks=8  per-chunk=[…]  agg=topk_mean → 0.901
```

Talking points: - Two models running in ensemble (HF + your trained CNN). - Per-chunk probabilities show the model's confidence over time. - Top-K mean catches localised fakery.

When the result loads, walk through: - The huge italic verdict — design language signals authority. - Confidence gauge. - **Per-chunk timeline** — point at the spike where the fake is most obvious. - **Mel-spectrogram with red overlay** — the model is highlighting suspicious windows.

Step 3 — Robustness test (1 min)

Click **+ Noise** in the robustness panel. Then **Telephone band**. Then **Pitch shift**.

Talking point: "The model's confidence drops here from 90 % to 72 % under telephone-band compression. This is exactly how a real deepfake survives in the wild — it gets re-encoded by Zoom, WhatsApp, or YouTube. Showing this on stage demonstrates we understand the limits."

Step 4 — Live mic recording (1 min)

Switch to the **Record** tab. Click the red record button. Speak for ~5 s ("Hi, my name is …, this is a real human voice"). Stop. Click **Submit**.

Talking point: "Detection runs on raw browser audio with no server-side gymnastics."

Step 5 — Side-by-side comparison (1 min)

Switch to **Compare**. Drop a real clip in A and a deepfake in B. Click **Run side-by-side**. Two cards appear with different verdicts.

Talking point: "Useful for forensic work — was this clip cloned from this other speaker?"

Step 6 — Feedback loop (2 min)

Re-analyse a clip the model got wrong (or one near the threshold). Scroll to the **Feedback** block. Click **Flag as FAKE** (or REAL).

Open the file explorer; show the new file in `feedback/fake/`.

Open the terminal:

```
python train.py --epochs 2 --feedback-only --limit 50
```

(Use `--limit 50` and `--epochs 2` for the demo so it finishes in a minute. For real retraining drop these.)

Talking point: "Active learning. Each flag is a labelled training sample. After enough have accumulated we re-train. The new model is automatically picked up by the ensemble next server restart."

Step 7 — Evaluation (1 min)

Open the terminal:

```
python evaluate.py --num 30 --ablate
```

Show the ablation table appearing at the end:

```
mode          accuracy         F1        EER
hf              91.67%     91.07%      6.67%
keras           96.67%     96.61%      3.33%
ensemble        96.67%     96.61%      3.33%
```

Talking point: "Reproducible benchmarks. We can quote real numbers in the report."

Step 8 — Out-of-distribution test (1 min, optional)

If time allows:

```
python evaluate_urls.py --ablate
```

Show how the same models perform on YouTube/archive.org clips — the gap between in-distribution and out-of-distribution accuracy is the single most important number in this project.

Closing (1 min)

> *"Three contributions: an ensemble pipeline that pairs a self-supervised pretrained model with a locally-trained CNN; a forensic UI with per-chunk visualisation, robustness probing, and human-in-the-loop feedback; and a reproducible evaluation harness."*

---

6. Likely questions in the viva (and honest answers)

Q. Why did you not just train a single model from scratch?

> *Self-supervised pretraining on millions of hours of speech captures structure that 13 K of our own data could never. The HF model is a faster path to generalisation. We kept the trained CNN because it's accurate on the in-distribution data and adds useful diversity in the ensemble.*

Q. Why does the model fail on YouTube clips?

> *Distribution shift. Training data is clean studio speech and a single TTS. YouTube has music, multi-speaker, codec compression. Mitigations: ensemble, top-K aggregation, voice activity detection on the roadmap.*

Q. What's EER and why do you report it?

> *Equal Error Rate — the threshold-free metric used by the ASVspoof community. It's the operating point where false-accepts and false-rejects are equal. It matters because in deployment you'd tune the threshold per use-case; EER is the threshold-independent benchmark.*

Q. How do you handle class imbalance?

> *Two ways: class-weighted binary cross-entropy in* `train.py` *(weights = total / 2·class_count), and decision-time we don't bias toward "fake" despite the 4:1 imbalance because the threshold is configurable.*

Q. What stops a user from flagging incorrectly to poison the model?

> *Right now nothing — it's a research tool. Production fixes would be: queue review by an admin, per-user rate limits, anomaly detection on flagged clips. The architecture supports that —* `feedback/` *is a staging area, not a direct training source.*

Q. Why top-K mean? Why not max?

> *Max is too noisy — a single rogue spike (e.g. silence chunk) flips the verdict. Top-K mean (default K=3) requires a small, sustained region of suspicion. Configurable via* `DEEPFAKE_AGGREGATION` *env var.*

Q. What's the saliency in the timeline chart?

> *Per-chunk gradient magnitude: `‖∂fake_logit/∂input‖`. It tells us which time regions the HF model is looking at most. Not a true Grad-CAM time-frequency heatmap (the HF model is a transformer over raw audio), but informative.*

Q. What would you do with more time?

> 1. *Voice activity detection (skip non-speech chunks).*
> 2. *AST (Audio Spectrogram Transformer) as a third ensemble member with proper time-frequency Grad-CAM.*
> 3. *Continual learning with replay buffer to avoid catastrophic forgetting on retrains.*
> 4. *Multi-language evaluation — test on Hindi/Tamil/Spanish to surface English-only bias.*

---

7. Key configuration knobs (for the report's "implementation details")

All env vars; set them before `python app.py` or before any of the eval scripts:

```
DEEPFAKE_MODEL                primary HF model id (default: motheecreator/Deepfake-audio-dete
ction)
DEEPFAKE_MODEL_2              optional second HF model (for HF×HF ensembles)
DEEPFAKE_KERAS_MODEL          path to .keras (default: model/fusion_model_final.keras)
DEEPFAKE_DISABLE_KERAS=1      force HF-only
DEEPFAKE_KERAS_WEIGHT=0.5     blend weight (0 = HF-only, 1 = Keras-only)
DEEPFAKE_THRESHOLD=0.5        decision threshold (lower = more sensitive)
DEEPFAKE_AGGREGATION          mean | max | topk_mean | p90  (default topk_mean)
DEEPFAKE_TOPK=3               K for topk_mean
DEEPFAKE_FAKE_INDEX           override label inversion (e.g. "0" or "model_id:1")
PORT=5001                     Flask port
INGEST_MAX_DURATION_S=1800    yt-dlp max audio duration
```

---

8. Files to know inside-out for the viva

If short on prep time, focus here:

1. **`detector.py`** — read top to bottom. Understand `_SingleModel`, `_KerasModel`, `Detector.predict`, `_aggregate`. This is the core of the project.

2. `train.py` — understand the `FusionSequence` data generator, the early-stopping callback, class weighting, versioned saves.
3. `app.py` — know all 8 endpoints and what they return.
4. `evaluate.py` — explain the ablation pattern (env var → `reload_detector()` → re-evaluate).

Don't bother memorising HTML/CSS/JS files — examiners rarely probe frontend code.

---

9. Quick reference — common commands

```
# run the app
python app.py

# offline eval on dataset/
python evaluate.py --num 60 --ablate

# OOD eval on URLs
python evaluate_urls.py --ablate

# fine-tune on accumulated feedback
python train.py --epochs 5

# retrain from scratch (long!)
python train.py --from-scratch --epochs 20

# train only on user-flagged clips (curriculum learning)
python train.py --feedback-only

# replace the bundled model with the new one (be careful)
python train.py --replace --epochs 5
```

---

10. Minimum knowledge checklist

Before the presentation, be able to answer **without notes**:

- [ ] What does Wav2Vec2 do?
- [ ] What does a mel-spectrogram represent?
- [ ] What's the difference between accuracy, precision, and EER?
- [ ] Why ensemble two models? When does it hurt?
- [ ] What is distribution shift and how do you mitigate it here?
- [ ] How does the feedback loop become training data?
- [ ] What's class weighting and why use it?
- [ ] How does chunked aggregation work? Why top-K mean?
- [ ] One concrete limitation of the system?

- [ ] One thing you'd do with another two weeks?

If you can answer all ten clearly, you'll do fine.

Good luck.